Research
Intelligent Manufacturing—Article

Deep Reinforcement Learning-based Multi-Objective Scheduling for Distributed Heterogeneous Hybrid Flow Shops with Blocking Constraints

Xueyan Sun^a, Weiming Shen^{a,*}, Jiaxin Fan^{a,*}, Birgit Vogel-Heuser^b, Fandi Bi^b, Chunjiang Zhang^a^a State Key Laboratory of Intelligent Manufacturing Equipment and Technology, Huazhong University of Science and Technology, Wuhan 430074, China^b Institute of Automation and Information Systems, Technical University of Munich, Garching 85748, Germany

ARTICLE INFO

Article history:

Received 16 November 2023

Revised 12 April 2024

Accepted 28 November 2024

Available online 20 December 2024

Keywords:

Multi-objective Markov decision process

Multi-agent deep reinforcement learning

Proximal policy optimization

Distributed hybrid flow-shop scheduling

Blocking constraints

ABSTRACT

This paper investigates a distributed heterogeneous hybrid blocking flow-shop scheduling problem (DHHBFSP) designed to minimize the total tardiness and total energy consumption simultaneously, and proposes an improved proximal policy optimization (IPPO) method to make real-time decisions for the DHHBFSP. A multi-objective Markov decision process is modeled for the DHHBFSP, where the reward function is represented by a vector with dynamic weights instead of the common objective-related scalar value. A factory agent (FA) is formulated for each factory to select unscheduled jobs and is trained by the proposed IPPO to improve the decision quality. Multiple FAs work asynchronously to allocate jobs that arrive randomly at the shop. A two-stage training strategy is introduced in the IPPO, which learns from both single- and dual-policy data for better data utilization. The proposed IPPO is tested on randomly generated instances and compared with variants of the basic proximal policy optimization (PPO), dispatch rules, multi-objective metaheuristics, and multi-agent reinforcement learning methods. Extensive experimental results suggest that the proposed strategies offer significant improvements to the basic PPO, and the proposed IPPO outperforms the state-of-the-art scheduling methods in both convergence and solution quality.

© 2024 THE AUTHORS. Published by Elsevier LTD on behalf of Chinese Academy of Engineering and Higher Education Press Limited Company. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

Intelligent manufacturing has become a major trend in the development of manufacturing industries where production scheduling is a key technology [1]. The hybrid flow-shop scheduling problem is widespread in various real-world industrial sectors such as the chemical, food processing, and steelmaking industries [2]. In some hybrid flow shops, there is no buffer zone between two successive stages because of special processing steps or process constraints. When a downstream machine is not available, a job that has finished on the current machine can only temporarily remain on that machine, and therefore blocks the next job process until the downstream machine is available. The blocking constraints increase the production line cycle time and the ineffective occupation rate of the machines. Therefore, studying the hybrid blocking shop scheduling problem is of great significance for

improving production efficiency and reducing production costs. When the number of machines is greater than five, the single-constraint blocking flow-shop scheduling problem (BFSP) has been shown to be a typical class of nondeterministic polynomial (NP)-hard combinatorial optimization problems [3]. Because the BFSP is a sub-problem of the distributed heterogeneous hybrid BFSP (DHHBFSP), it is also an NP-hard optimization problem.

Distributed manufacturing enables the efficient distribution of raw materials and an optimal combination of productivity to achieve rapid manufacturing of high-quality and low-cost products [4]. The hybrid flow-shop scheduling problem combines both distributed manufacturing and blocking constraints and can be defined as the distributed hybrid BFSP (DHHBFSP) [5]. Generally, in a DHHBFSP, manufacturing tasks are collaboratively performed by several identical blocking floor shops. However, in real distributed manufacturing problems, the number of machines and machine processing times in shops vary from place to place owing to different production conditions, resulting in different energy consumption [6]. Additionally, in mass personalization, jobs with heterogeneous requirements arrive dynamically over time, requir-

* Corresponding authors.

E-mail addresses: wshen@ieee.org (W. Shen), fanjx@hust.edu.cn (J. Fan).

ing scheduling decisions to be made on the fly. Therefore, studying the dynamic DHHBFSP is more relevant to the manufacturing industry.

Increased energy consumption is a major global concern. Improving energy efficiency and achieving the goal of low-carbon development should focus on solving bottlenecks in production operations [7]. Iron and steel production is typically a blocking flow-shop manufacturing process that consumes significant energy. The iron and steel industry employs high-temperature furnaces for iron and steel production and has become the second-largest energy consumer in the industry [8]. In 2013, the world's total final industrial energy consumption was 113 131 PJ, of which the iron and steel sector accounted for 18% [9]. Hernandez et al. [10] summarized the energy intensity, exergy intensity, and resource efficiency results for three steelmaking routes and nine individual plants. The actual resource efficiency of global steel production is only 32.9%, owing to significant energy losses. Similarly, the manufacturing of pharmaceutical products involves a variety of chemical reactions and syntheses, which are associated with high energy consumption [11].

The distributed BFSP (DBFSP) is mainly solved using exact methods [12], heuristic rules [13], and meta-algorithms [5]. Exact methods can obtain optimal solutions, but are not applicable to large-scale problems. Single heuristic rules are efficient but produce poor results for large-scale problems. Although metaheuristics can obtain better scheduling solutions, their responsiveness is worse for dynamic production states and they are time-consuming compared with heuristic rules. Reinforcement learning (RL) combined with intelligent agents, can efficiently use real-time data from the production process to make effective production decisions and respond quickly to dynamic changes. Riedmiller and Riedmiller [14] was the first to introduce RL to the field of dynamic scheduling, which allowed an agent to adaptively select the priority dispatching rule (PDR) that is most beneficial for the scheduling goal. This composite dispatching rule approach improves the dispatching performance of the PDR to a certain extent but reduces the solution space for a complex and variable production scheduling problem and does not result in a better dispatching solution.

Our motivation for this work is to investigate the DHHBFSP with both productivity- and energy-related objectives and develop an end-to-end deep RL (DRL)-based scheduling method to make real-time decisions. Therefore, this study formulates a multi-objective Markov decision process (MOMDP) model for the DHHBFSP and proposes an improved proximal policy optimization (IPPO) method to address this problem. The novelty and main contributions of this study are as follows:

- (1) An MOMDP model is established for the DHHBFSP by defining the state features, a vector-based reward function, and the end-to-end action space.
- (2) A multi-agent DRL (MADRL) framework is adopted for a distributed manufacturing environment, where a factory agent (FA) is equipped for each hybrid blocking flow shop, and multiple FAs work asynchronously to select unscheduled jobs.
- (3) An IPPO method is proposed to train a single agent that incorporates two proximal policy optimization (PPO) networks with different weights to explore additional Pareto solutions.
- (4) A two-stage training strategy is developed to improve data utilization by training with both single- and dual-policy data.

The remainder of this paper is organized as follows. Section 2 provides a literature review. Section 3 describes the problem and establishes a mathematical model for a multi-objective DHHBFSP. Section 4 elaborates on the MOMDP formulation and the proposed IPPO. Section 5 presents the numerical experimental results and

analyses. Finally, Section 6 summarizes the study and discusses future research directions.

2. Related Literature

2.1. DBFSP

For the DBFSP, Ribas et al. [12] first developed a mixed-integer programming model with makespan as an optimization objective to solve small-scale instances. To solve large-scale instances, they designed a hybrid iterative greedy algorithm with variable neighborhoods and iterative local search algorithms. Zhang et al. [15] proposed two different mathematical models to calculate the makespan and a hybrid discrete differential evolution algorithm to deal with the DBFSP. For the DBFSP with the total flowtime criterion, Chen et al. [16] took advantage of both the population-based search approach and iterated greedy algorithm to generate offspring solutions and enhance the local exploitation ability. Shao et al. [5] first attempted to solve the DBFSP using a fruit fly optimization algorithm and proposed a hybrid enhanced discrete fruit fly optimization algorithm, including effective initialization schemes based on scent- and vision-based foraging. In the same year, they [13] studied a distributed fuzzy blocking flow-field scheduling problem, and proposed two constructive heuristics based on problem-specific knowledge and Nawaz–Enscore–Ham (NEH) heuristics. Subsequently, they presented a mixed-integer linear programming model to formulate a heterogeneous DBFSP, and proposed a learning-based-selection hyperheuristic framework for solving it [17]. Qin et al. [6] proposed a mathematical model of a DHHBFSP, and designed a collaborative iterative greedy algorithm. Zinn et al. [18] applied a deep Q-learning method to make decisions in a case study of a blocking flow shop. Ren et al. [19] studied the DPFSP using Nash Q-learning and obtained better results than the metaheuristics. Yang et al. [20] studied a DPFSP with dynamic job arrivals using DRL to select rules.

Most studies on energy-aware flow-shop scheduling problems consider the energy consumption as an additional objective to productivity-related metrics, and adopt multi-objective optimization frameworks for the tradeoff. Chen et al. [21] considered the effect of machine processing speed on energy consumption and proposed a collaborative optimization algorithm for a distributed no-idle flow-shop scheduling problem. Zhang et al. [22] investigated the impact of production scheduling decisions aimed at improving production and energy performance simultaneously in distributed blocking flow shops. A Bayesian inference-based probabilistic model and specific speed adjustment operators were proposed to obtain better search spaces for both objectives. Mou et al. [23] studied the inside plant of distributed flow shops with energy consumption, proposed a knowledge-driven solution strategy based on machine learning, designed a hybrid collaborative algorithm and dual-population collaborative search mechanism, and achieved a balance between global exploration and local development. Zhao et al. [24] presented a hyperheuristic with Q-learning to address the energy-efficient DBFSP, where Q-learning was employed to select an appropriate pre-designed low-level heuristic. Shao et al. [25] proposed some constructive local search methods selected by Q-learning to address an energy-efficient distributed fuzzy hybrid BFSP. Zhao et al. [26] combined the neighborhood perturbation operator and Q-learning algorithm to select the appropriate perturbation operator in the search process. Bao et al. [27] developed a top-level Q-learning model for improving machine utilization by finding scheduling policies from four sequence-related operations, as well as a bottom-level Q-learning

model for improving energy efficiency by learning the optimal speed-governing policy. Most research efforts have been devoted to selecting operators or dispatch rules by Q-learning, whereas applications of end-to-end DRL in DBFSP are limited.

2.2. Multi-objective RL (MORL)

RL was developed based on the Markov decision process (MDP). MORL can be viewed as a combination of multi-objective optimization and RL techniques to solve sequential decision-making problems with multiple conflicting objectives [28]. MORL is an MDP in which rewards are vectors (instead of scalars) whose components, called objectives, are interpreted as different criteria. There have been several studies on MORL. Gábor et al. [29] assumed a fixed order between different rewards, where the sub-objectives of the problem were considered constraints. Techniques based on formulating a multi-reward problem by optimizing the weighted sum of the discounted total rewards for multiple reward types were presented by Feinberg and Schwartz [30]. Russell and Zimdars [31] decomposed the reward function into multiple components learned independently (with a single policy). Barrett and Narayanan [32] defined the convex hull as the smallest convex set that contains all the sets of points that lie on the boundary of a convex set, which are the extreme points that are maximal in some direction. This is similar to the Pareto curve, because both are maxima over the tradeoffs in linear domains. Instead of updating a single Q-value as the sum of the weighted objectives, they updated a set of Q-values in the convex hull. Van Moffaert and Nowé [33] proposed a Pareto Q-learning algorithm with three mechanisms that allowed for action selection based on the content of the sets of Q-vectors. By storing the average observed immediate reward and the set of nondominated vectors in the next state separately, they also allowed them to converge separately.

With the development of deep learning, many scholars have focused on its application to MORL. Mossalam et al. [34] proposed deep optimistic linear support (OLS) learning to solve high-dimensional multi-objective decision problems, where the relative importance between the objectives is not known a priori. This was the first time that deep learning was introduced into MORL. OLS takes an outer-loop approach, in which the convex coverage set is incrementally constructed by solving a series of scalarized Markov decision processes (MDPs) for different linear scalarization weight vectors. Abels et al. [35] proposed a conditioned network in which a Q-network was augmented to output weight-dependent multi-objective Q-value-vectors. To train this network efficiently, they used an update rule to set the dynamic weights and a diverse experience replay to improve sample efficiency and reduce replay buffer bias. Nguyen et al. [36] introduced a scalable multi-objective DRL framework based on deep Q-networks (DQNs) that supports both single- and multi-policy strategies, as well as both linear and nonlinear approaches to action selection. Siddique et al. [37] used a nonlinear generalized Gini social welfare function to balance the objectives, instead of the usual linear weighted approach to action selection. He et al. [38] addressed the multi-objective optimization problem of the textile manufacturing process using a DQN-based multi-agent RL (MARL) system. They proposed a self-adaptive DQN-based MARL framework in which m_A ($m_A \in \mathbf{N}^+$) optimization objectives are formulated as m_A DQN agents that are trained through a self-adaptive process constructed using a Markov game. Although multi-objective deep RL (MODRL) has developed significantly, there have been few studies on floor-shop scheduling problems. Yang et al. [39] used vectorized value functions and performed envelope updates by utilizing the convex

envelope of the solution frontier to update the parameters. Luo et al. [40] developed a two-hierarchy DQN to address the multi-objective flexible job shop problem with new job insertions. A higher DQN is a controller that determines the optimization goal for the lower agent, serves as an actuator, and outputs the Q-value of each dispatching rule. This method does not consider multiple objectives simultaneously. Therefore, an effective MODRL method for scheduling should be explored.

2.3. Research gaps

According to the aforementioned literature, it can be concluded that: ① DRL methods are usually applied to select operators for metaheuristics, which cannot respond to random job arrivals in real time; ② a DRL agent can be trained to select dispatch rules for better responsiveness, but the solution quality is not satisfactory; and ③ end-to-end DRL has shown the potential for representing the vast action space in scheduling problems. To the best of our knowledge, end-to-end DRL has not been applied to distributed hybrid blocking flow-shop scheduling. Because the DHHBFSP includes even more complex characteristics and requires real-time decisions, this study considers the development of an end-to-end DRL-based online scheduling method that directly selects unscheduled jobs according to the probability distribution obtained by agents.

From the perspective of policy networks, DRL methods can be classified into single- and multi-policy methods. A single policy allocates weights to multiple objectives and merges them into a single objective for evaluation. Multi-policy methods can fully explore the convex hull for small-scale continuous optimization problems and have also been adapted to explore Pareto fronts in discrete optimizations, such as scheduling problems. However, the training time increases exponentially owing to the complexity of the state space. Therefore, this study adopts a single-policy framework and incorporates two PPO networks in a single agent, where a vector-based reward function is developed to address the intractable issue of setting the weights of different objectives.

3. Problem descriptions

The DHHBFSP addressed in this paper can be described as follows:

There is a set of jobs $J = \{J_1, J_2, \dots, J_i\}$ that randomly arrives at the enterprise and must be allocated to F different hybrid flow factories $H = \{H_1, H_2, \dots, H_F\}$. All jobs have K operations with the same processing routes. However, the processing time depends on the assigned factories. For example, the standard processing time for the j th operation of J_i is represented as p_{ij} ($i, j \in \mathbf{N}^+$). Then, when J_i is assigned to H_f ($f \in \mathbf{N}^+$) and processed at the j th stage with the processing factor μ_{fj} , the actual processing time should be $p_{ij} \cdot \mu_{fj}$. A hybrid flow shop H_f contains K processing stages and at least one stage is equipped with more than one available machine, where the number of machines at the same stage, denoted by M_{fj} , can vary from different factories. In addition, a blocking constraint is considered in this environment, which means that a job can only be released if a machine is available in the next stage. Otherwise, it should be held at the current machine; such a blocking state consumes a certain amount of energy. Note that the total number of jobs is unknown for the DHHBFSP because of the randomness of the job arrivals. However, to simplify the mathematical formulation, it is assumed that there N jobs exist during the entire scheduling horizon. The other common production constraints are as follows:

- (1) All machines are available at time zero.
- (2) A job can only be processed by one factory.
- (3) A job can only have one operation processed at a time.
- (4) A machine can process only one operation at a time.
- (5) All setup and transportation times are neglected or included in the processing time.

Therefore, the DHHBFSP comprises two decision-making processes: allocating unscheduled jobs to factories and determining the processing sequence for each factory. The objective of the DHHBFSP is to minimize the total tardiness (TT) and total energy consumption (TEC) simultaneously. Once a feasible schedule for the DHHBFSP is given, the TT can be easily identified according to the completion time and due dates, whereas the TEC includes the energy consumption during the processing, idle, and blocking states. Here are the calculation of TT and TEC.

$$\text{Minimize TT} = \sum_{i=1}^N \max(c_{i,K} - D_i) \quad (1)$$

$$\text{Minimize TEC} = E_p + E_i + E_b \quad (2)$$

$$E_p = \sum_{i=1}^N \sum_{j=1}^K \sum_{f=1}^F x_{if} \cdot p_{ij} \cdot \mu_{fj} \cdot e_{fj} \quad (3)$$

$$E_i = \sum_{i=1}^N \sum_{j=1}^K \sum_{f=1}^F x_{if} \cdot \alpha_{ij} \cdot e_{fj} \cdot e_{\text{idle}} \quad (4)$$

$$E_b = \sum_{i=1}^N \sum_{j=1}^K \sum_{f=1}^F x_{if} \cdot \beta_{ij} \cdot e_{fj} \cdot e_{\text{block}} \quad (5)$$

where F is the total number of hybrid flow factories, D_i is due date of J_i , $c_{i,K}$ is completion time of the K th operation of J_i , α_{ij} is the idle time of J_i at the j th operation, β_{ij} is the blocking time of J_i at the j th operation, μ_{fj} is the process time factor of operations at the j th stage in factories H_f , e_{fj} is the standard unit time energy consumption at the j th stage in factories H_f , e_{idle} is the factor of unit energy consumption during the idle time of machines, e_{block} is the factor of unit energy consumption during the blocking time of machines. $x_{if} = 1$, if J_i is assigned to H_f , otherwise, $x_{if} = 0$. TT is the total tardiness, TEC is the total energy consumption, E_p is the energy consumption during the processing state, E_i is the energy consumption during the idle state, and E_b is the energy consumption during the blocking state.

This work also establishes a mathematical model to describe the multi-objective DHHBFSP. This model can be found in [Part A](#) in Appendix A.

4. PPO-based method for DHHBFSP

4.1. Preliminary studies

The proposed IPPO aims to address the multi-objective DHHBFSP by improving the PPO training method using a multi-agent framework. Therefore, this subsection introduces some preliminaries of the proposed IPPO for better understanding, including MORL and MADRL.

4.1.1. MORL

In RL, the environment is typically modeled as a MDP that contains $M = (S, A, P, \gamma, R)$ with five parts. S is a set of states s in the environment, A is a set of actions a for agents, P is the transition

function $P(s'|s, a)$ denoting the probability that the agent finds itself in each possible next state s' after executing action a in state s , a reward function $R(s, a)$ denoting the immediate reward obtained by executing action a in state s . $\gamma \in [0, 1]$ is the discount factor. In multi-objective optimization, the objective space consists of two or more dimensions. Therefore, regular MDPs are generalized as MOMDPs. MOMDPs are MDPs that provide a vector of rewards instead of a scalar reward, that is, $\mathbf{R}(s_i, a_i) = (R_1(s_i, a_i), R_2(s_i, a_i), \dots, R_m(s_i, a_i))$, where m represents the number of objectives ($m \in \mathbf{N}^*$). In MORL, the state-dependent value function $\mathbf{V}$ of state s is a vector.

Single-policy MORL algorithms employ scalarization functions to define utility over a vector-valued policy, thereby reducing the dimensionality of the underlying multi-objective environment to a single scalar dimension. The scalarization function g projects the vector $\mathbf{V}$ to a scalar:

$$\mathbf{V}_w = g(\mathbf{V}, \mathbf{w}) \quad (6)$$

where $\mathbf{w}$ is the weight vector that parameterizes g . A scalarization function can come in many forms, however, the most common function is the linear scalarization function, which is used in this study.

4.1.2. Multi-agent DRL

Typical MADRL algorithms such as multi-agent deep deterministic policy gradient (MADDPG) [41] and multi-agent PPO (MAPPO) [42] perform centralized training and distributed execution (CTDE). CTDE means that the networks of all agents need to be trained together with joint states and actions (centralized training), but each agent can make independent decisions and output its own actions (distributed execution). Because each agent must consider the actions and states of other agents when making decisions, frequent data interactions are required between the agents. When individual agents are trained centrally, their critic networks are meant to use the global state; therefore, when the number of agents increases, learning becomes increasingly difficult to scale, which is also a challenge for the training cost and convergence of the algorithm. At the same time, not all agents need to make decisions at the same moment, thus, if a synchronized decision measurement is used, it may produce a large amount of invalid data, which reduces the training efficiency.

Therefore, this study adopts a multi-agent DRL approach with asynchronous decision making, where multiple agents collaborate and influence each other through rewards. The agents are trained separately and executed asynchronously. When the simulation environment runs to the time of the first stage of the current job departures for one agent (decision time), the next job is selected from the job pool, and the reward generated after the selection is added to the state of the next decision point. Thus, the actions performed by different agents before and after the selection of the jobs affect each other. Therefore, multiple agents are asynchronous in decision-making, but collaboration between agents is achieved through the change in the state of the jobs and rewards.

4.2. Formulation of the MOMDP

4.2.1. State

The state features designed in this study consist of vectors of the state features of the jobs to be processed, and all features reflect the current global state of the factory. According to the MDP definition, the next decision is related only to the current state of the environment and is independent of the decisions made previously. Therefore, the state features of each job must contain only the processing information for unprocessed operations. Every agent makes decisions based on local observations, including the

Table 1
State definitions.

State	Definition
Process times	All process times on all stages
D-A	Time difference between the due date and current time
Blocking time	Estimated blocking time after choosing this job
Tardiness signal	Equal to 1 if no tardiness, otherwise equal to -1
Reward of last decision	Reward vector of last decision
Job type of last decision	Job from new arrival pool or not
Estimated immediate reward	The estimated immediate reward after choosing this job
Weights of all objectives	Weights between two objectives

state vectors of L jobs. Each vector contains 14 features that describe the state of the current manufacturing environment. The details of the state vectors are presented in Table 1. The state vector for one agent is shown in Fig. 1. State transfer was performed using the Python SimPy simulation library in a multi-agent environment.

4.2.2. Action

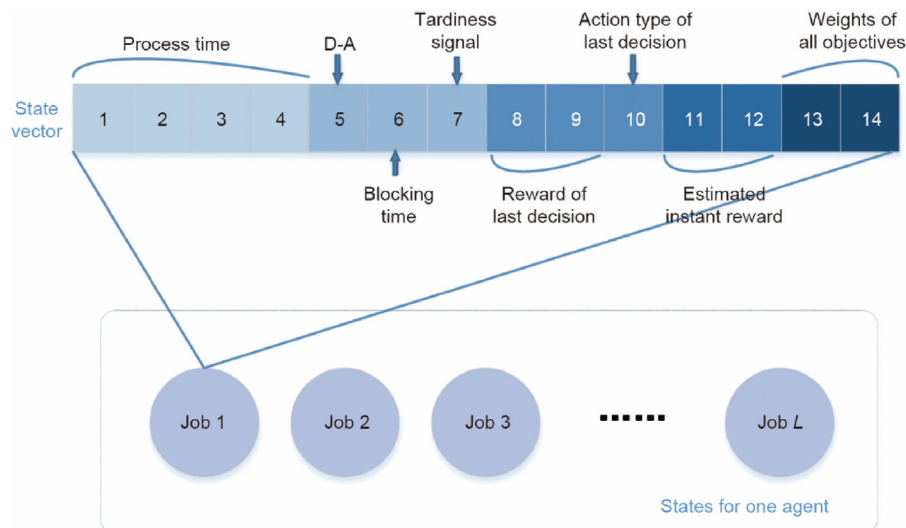
To improve training efficiency, each episode has initial jobs, and a scheduling plan is generated according to the hybrid genetic algorithm (HGA). Additionally, some jobs arrive randomly. New jobs do not select a factory autonomously but are placed in the job pool with all unprocessed jobs. The selected job is directly decoded and arranged into the corresponding factory stages, and machine selection is performed according to the first available machine (FAM) rules. The departure time of each stage is updated after decoding, and when the simulation environment runs to the departure time of the first stage of the current job of one agent (decision time), the next job is selected from the job pool. The action in our MOMDP is to choose a job from the current job pool. The number of jobs in the job pool changes with the process time. To fix the input size, this study sorts all jobs and selects eight jobs using the earliest due date (EDD) rule for the agent to make a decision.

4.2.3. Reward

In this study, reward $\vec{R}$ is a vector instead of a scalar and corresponds to the two objectives. It comprises three parts: immediate reward $\vec{IR}$, episode reward $\vec{ER}$, and previous reward $\vec{PR}$. To avoid obtaining single-step rewards during the learning process of the agent, $\vec{IR}$ and $\vec{ER}$ functions were designed separately. $\vec{IR}$ is a vector of negative tardiness and negative energy consumption generated by the job selection. The actual average tardiness and energy consumption for all factories in each episode constitute the $\vec{ER}$ and are updated to the rewards for all agents at the end of each episode. The effect of the previous decision on the current state was considered by adding a $\vec{PR}$ with a discount factor. To avoid the negative impact of excessively large or small reward values on the training of the state-value function, this study applies the reward scaling method [43] to adjust the scale of the rewards.

$$\vec{IR} = \left(\frac{1}{T_t}, \frac{1}{EC_t} \right) \quad (7)$$

$$\vec{PR} = \left(\sum_{i=0}^{t-1} \gamma \frac{1}{T_i}, \sum_{i=0}^{t-1} \gamma \frac{1}{EC_i} \right) \quad (8)$$

**Fig. 1.** States for one agent ($L \in \mathbb{N}^+$).

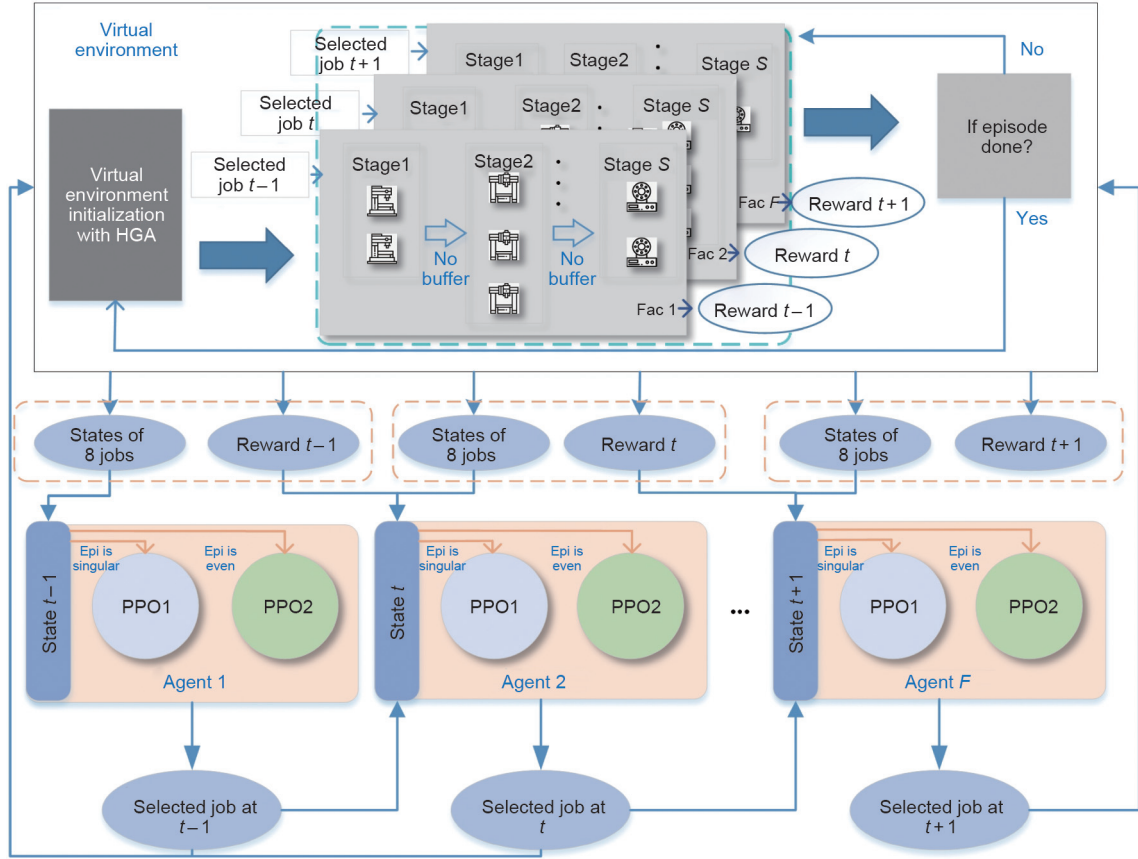


Fig. 2. Multi-agent training framework. Fac: factory; Epi: the number of the episodes.

$$\vec{ER} = \left(\frac{1}{TT \sum \text{totalsteps}_f}, \frac{1}{TEC \sum \text{totalsteps}_f} \right) \quad (9)$$

$$\vec{R} = \vec{IR} + \vec{PR} + \vec{ER} \quad (10)$$

where T_t is the tardiness at time t , EC_t is the energy consumption at time t , γ is a discount factor, totalsteps_f is the number of the steps for factory f in each episode.

4.3. Training process based on PPO

4.3.1. MADRL-based training framework

In this section, we introduce the MADRL-based training framework for the proposed IPPPO. In the DHHBFSP environment, a DRL agent is formulated for each factory and defined as an FA for job allocation. Multiple FAs work in an asynchronous manner without direct interaction, which means once an FA has selected a job and updated the schedule, other FAs will observe an updated state that includes such changes to avoid conflicts. The proposed multi-agent training framework is shown in Fig. 2. Each agent trains the networks independently; however, the action and reward of the previous step in the entire environment are considered as the states of the next step. The multi-agent virtual environment is initialized by a fixed number of jobs distributed by the HGA proposed in a previous study [44]. Before new jobs arrive, all agents process

the jobs in the sequences produced by the HGA. Once a new job arrives, all agents transfer to a learning process and must choose the next job at decision points with PPO networks until there are no jobs for all agents to choose. After an episode is completed, the virtual environment is reinitialized for a new episode. This process ends when a predefined number of episodes is reached.

Based on this training framework, we propose a training method for MODRL. Prior to the training process, the parameters of the environment and training are set. During each episode, the virtual environment is reset with 20 random initial jobs distributed by the HGA to different agents for processing. After new jobs arrive, when reaching a decision point, agent i selects eight jobs from the job pool with the EDD as the input and obtains observations o_i and masks m_i . As mentioned previously, each agent has two PPO networks: $PPO_{i,1}$ and $PPO_{i,2}$. If the episode is an odd number, agent i chooses actions by $PPO_{i,1}$, receives reward r_i , next observation o'_i , and next mask m'_i , and stores the tuple $(o_i, m_i, a_i, r_i, o'_i, m'_i)$ into $\text{memory}_{i,1}$. Otherwise, agent i chooses actions using $PPO_{i,2}$. In the learning process, if the episode reaches half of the training iterations for policy (TR), each network samples a continuous batch of memories to update the network parameters and retain the memories. If the episode reaches the TR, each network samples a continuous batch of memories to update the network parameters and empties the memories. The training process is terminated when the maximum number of episodes M is reached. The MODRL training process is presented in Algorithm 1.

Algorithm 1. Training process of the proposed IPPPO.

```

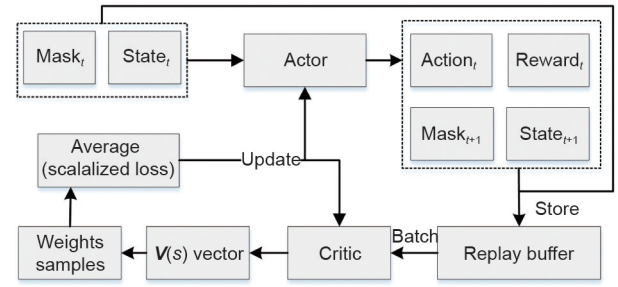
1  for episode = 1 to  $M$  do
2    Reset the training environment with 20 initial jobs
    randomly
3    Generate an initial schedule by the HGA
4    for  $f = 1$  to  $F$  do
5      Combine all unscheduled jobs and select 8 jobs by
      the EDD
6      The  $f$ th agent obtains the observation
       $o_i = [o_1, o_2, \dots, o_5]$  and mask  $m_i$ 
7      if episode%2 = 1 then
8        Choose action  $a_i$  for PPO $f,1$ 
9        Obtain reward  $r_i$ , the next observation  $o'_i$  and the
        next mask  $m'_i$ 
10       Decompose the action for the machine selection
11       Store the tuple  $(o_i, m_i, a_i, r_i, o'_i, m'_i)$  into memory $f,1$ 
12       if episode%TR = TR/2 then
13         PPO $f,1$  trained with a random  $B$  of memory $f,1$ 
14         Preserve the memory and update the network of
        PPO $f,1$ 
15       else if episode%TR = 0 then
16         PPO $f,1$  trained with a random  $B$  of memory $f,1$ 
17         Empty memory $f,1$ 
18       end if
19       else if episode%2 = 0 then
20         Choose action  $a_i$  for PPO $f,2$ 
21         Obtain reward  $r_i$ , the next observation  $o'_i$  and the
        next mask  $m'_i$ 
22         Decompose the action for the machine selection
23         Store the tuple  $(o_i, m_i, a_i, r_i, o'_i, m'_i)$  into memory $f,2$ 
24         if episode%TR = TR/2 then
25           PPO $f,2$  trained with a random  $B$  of memory $f,2$ 
26           Preserve the memory and update the network of
        PPO $f,2$ 
27         else if episode%TR = 0 then
28           PPO $f,2$  trained with a random  $B$  of memory $f,2$ 
29           Empty memory $f,2$ 
30         end if
31       end if
32     end for
33  end for

```

4.3.2. PPO-based training in a single agent

In this study, the PPO training framework is used to train a deep learning network [45]. Generally, each agent trains only one PPO network. However, training using only one network may reduce the solution range. Therefore, we train two PPO networks on an agent with different objective weight distributions. During the training process, these two PPO networks store the training data separately according to the odd and even numbers of episodes. The training processes for the two PPO algorithms are identical, as shown in Fig. 3.

As mentioned previously, this is a multi-objective problem; the reward and state value functions should be vectors representing the two objectives. During the training process, the loss values of the actor and critic networks must be scalar to update the deep network parameters. Instead of using different linear scalarization weight vectors to solve a series of scalarized MDPs, in this study, 50 sets of weight vectors $[w_{1,i}, w_{2,i}]$ are used to calculate the average scalarized loss value, as shown in Eq. (11) loss vector $[l_1, l_2]$.

**Fig. 3.** Training process in single PPO. $V(s)$: the state-dependent value with state s .

$$\text{loss} = \sum_i w_{1,i} l_1 + w_{2,i} l_2 \quad (11)$$

The PPO algorithm trains the actor and critic once using the same training data TR times. To use data more efficiently, this study proposes an improved learning strategy that divides the training process into two parts: single-policy and dual-policy data training. For the first $\frac{TR}{2}$ times, the networks are updated based on data from the old policy. Subsequently, the networks produce another set of training data with an updated policy and update the current networks by combining these two sets of data for the remaining $\frac{TR}{2}$ times.

In this study, we adopted the attention-based policy network (APN) proposed in Ref. [46], which mainly consists of a feature extraction layer, query glimpse layer, and pointer layer. The input of the APN is the proposed job vector and the output is the probability distribution of the input jobs. The details of the APN can be found in Part B in Appendix A.

5. Numerical experiments**5.1. Experiment setups**

All the compared methods were coded in Python and performed on a computer with an Advanced Micro Devices (AMD) Ryzen 56 600H (American Technology Company Inc., USA) with a Radeon Graphics Central Processing Unit (CPU) at 3.30 GHz (American Technology Company Inc., USA) with 16 GB of Random Access Memory (RAM; American Technology Company, Inc., USA) running the Windows 10 Standard 64 bits operating system (Microsoft, USA). Before the training process, the layout of the multi-factory environment was defined as shown in Table 2. The running environment for the DHHBFSP provides open access to GitHub.

5.1.1. Setting of test instances

The input was the states of eight jobs containing eight vectors in 14 dimensions. The output for the actor was eight actions and the

Table 2
Layout of the multi-factory environment.

Setting	Values
The number of initial jobs	20
The number of factories	2
The number of stages	4
The number of machines in factory 1	[2, 3, 1, 4]
The number of machines in factory 2	[4, 3, 3, 2]
Time factors of factory 1	[1, 1.3, 0.7, 1]
Time factors of factory 2	[1.3, 1, 1, 0.7]
Energy consumption factors of factory 1	[1.3, 0.7, 1.3, 0.7]
Energy consumption factors of factory 2	[0.7, 1, 1, 0.7]
Energy consumption factors of blocking state	0.3
Energy consumption factors of idle state	0.2

output for the critic was two Q values. The dimensions of both the hidden and embedded layers were set to 128. There are eight heads in the multi-head attention layer. The Adam optimizer was employed for parameter optimization. The initial learning rate was set as 1×10^{-5} . For new jobs, we assume that the job arrival pattern follows a Poisson distribution with arrival rate λ , according to queuing theory. Instances of different scales were randomly generated for the experiments. Then, the number of total jobs was set as n total $\in \{10, 20, 50, 100, 200\}$. The due date for these jobs was calculated by summing the process times with different due date tightness (DDT) $\in \{1, 1.5\}$. There were five examples for each number, for a total of 50 instances.

5.1.2. Performance metrics

Before the experiments, two indicators were defined to evaluate the performance: invert generational distance (IGD) and purity (P). The IGD is a measured value defined as the distance between each point in the optimal Pareto front (PF*) and the obtained Pareto front for an algorithm (PF). IGD is the variation in generational distance (GD) and is a more comprehensive indicator that reflects convergence and diversity simultaneously. It is shown as:

$$IGD = \frac{\sqrt{\sum_{i=1}^n d_i^{*2}}}{n} \quad (12)$$

where d_i^* denotes the Euclidean distance between each point consisting of the PF* and the nearest points of the PF, n is the number of PF* points. Therefore, a low IGD value is desirable. Because the true PF* may be unknown or difficult to obtain, in this study, PF* was obtained using all runs of the different approaches for each instance. P is the ratio of nondominated solutions generated by an algorithm X after comparison with the nondominated solutions generated by other algorithms. The P metric reflects the cardinality of an algorithm. It has a value range of $[0, 1]$, where a value of 1 indicates that the nondominated set ND_X completely dominates all solutions generated by other algorithms. Thus, a higher value

of P is desirable, and the formulation of P in algorithm X is described as

$$P_X = \frac{|ND'_X|}{|ND'|} \quad (13)$$

where $|ND'_X|$ is the number of nondominated solutions generated by algorithm X after comparison with other nondominated solution sets, and $|ND'|$ is the number of solutions of all nondominated solution sets among the compared algorithms.

5.1.3. Parameter tuning

The proposed IPPO contains four key parameters: B (batch size), γ , previous reward discount factor (PR), and training iteration for policy (TR). To take advantage of the IPPO and reduce the influence of the parameters on different instances, the Taguchi method, which is one of the best statistical analysis methods with fewer experiments, was adopted for several selected instances.

The orthogonal array L9(3^4) is designed. Table 3 lists the values of each parameter. Each combination will run 1000 episodes and solve a random instance 30 times independently. From Fig. 4, we can see that the best parameter values for B , γ , PR, and TR are 1024, 0.95, 0.9, and 40, respectively. Furthermore, batch size B is the most sensitive to the training process. γ shows the least impact on the training process. The response values for these parameters can be found in Part C in Appendix A.

Table 3
Levels of parameters.

Level	B	γ	PR	TR
1	256	0.85	0.7	40
2	512	0.90	0.8	60
3	1024	0.95	0.9	80

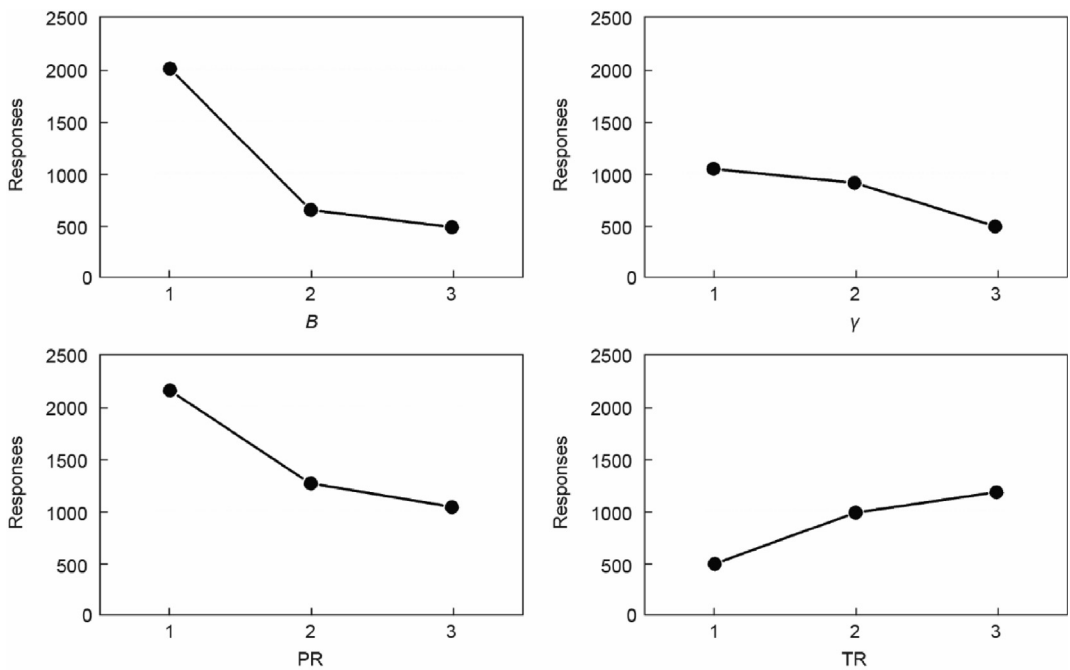


Fig. 4. Factor level trend of the IPPO.

5.2. Comparison with PPO variants

The training process is one of the most important components of DRL. Two experiments were conducted to demonstrate the effectiveness of the proposed training method. First, the training process with a single PPO network was compared to that with dual PPO networks. Second, the training framework with the new learning strategy was compared to the framework without the new learning strategy. Each instance was solved 30 times independently. Detailed results can be found in [Part D](#) in Appendix A.

5.2.1. IPPO vs PPO_S: Validation of dual PPO networks

In the proposed IPPO method, two PPO networks were used to search for solutions in different spaces. Therefore, we assigned different pairs of weights to each input. In addition, to simplify the training process, we placed the replay data in two buffers. In contrast, the input of the training process with a single PPO network (PPO_S) does not assign a set of weights, and the replay buffer requires only one.

[Fig. 5](#) shows the convergence of the two methods, and indicates that the PPO_S can reach the convergence state at approximately

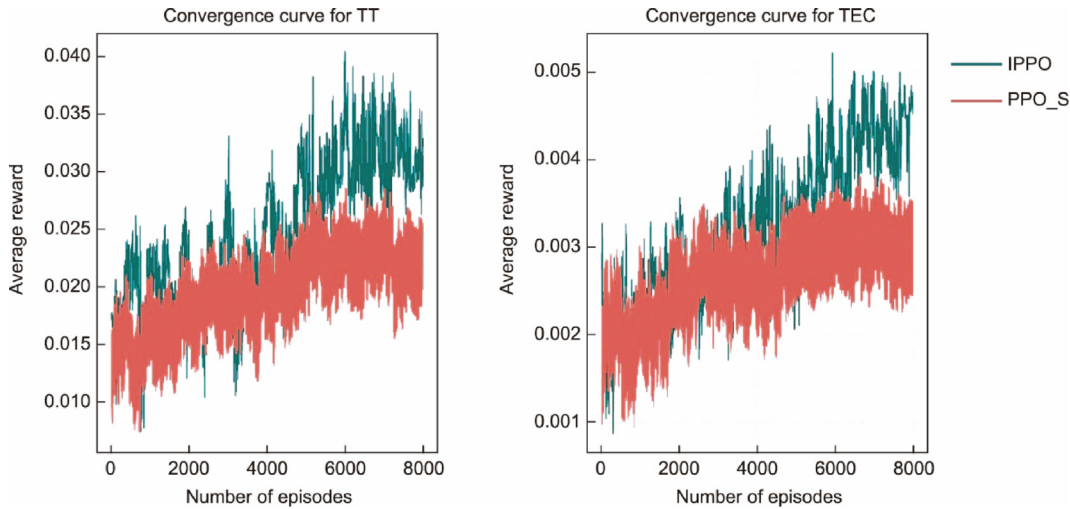


Fig. 5. Convergence curves of IPPO and the PPO_S.

Table 4
IGD values and *P* values between IPPO and PPO_S.

Instance	IGD (DDT = 1)		<i>P</i> (DDT = 1)		IGD (DDT = 1.5)		<i>P</i> (DDT = 1.5)	
	PPO_S	IPPO	PPO_S	IPPO	PPO_S	IPPO	PPO_S	IPPO
J10	5.61×10^3	5.57×10^2	0	0.51	5.50×10^3	9.88×10^1	0	0.68
J20	6.97×10^3	3.04×10^2	0	0.75	5.48×10^3	4.67×10^2	0	0.72
J50	4.38×10^3	7.91×10^2	0.03	0.53	8.98×10^3	3.19×10^2	0	0.83
J100	4.96×10^3	2.85×10^3	0.23	0.36	1.29×10^4	2.67×10^3	0	0.72
J200	2.19×10^4	1.37×10^4	0.18	0.58	9.62×10^4	3.77×10^3	0.05	0.74

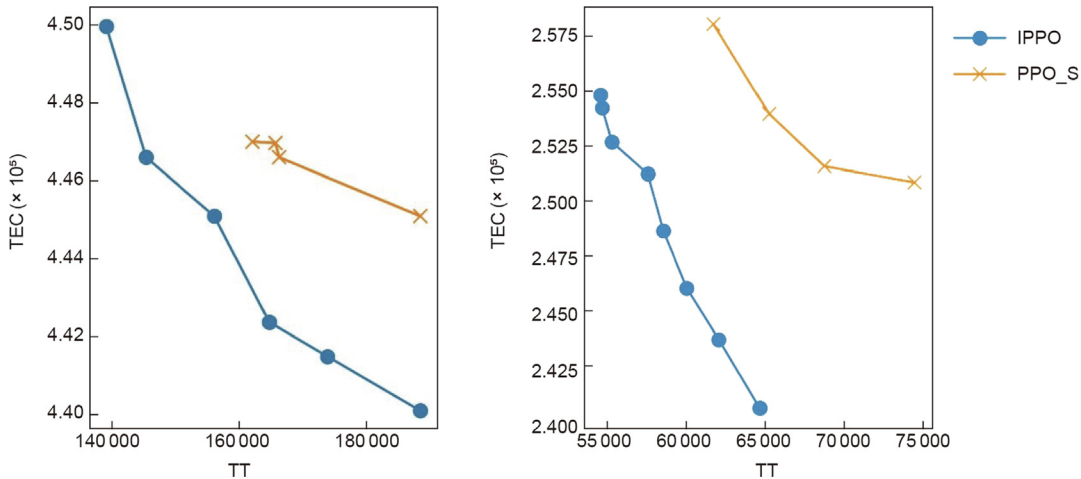


Fig. 6. Pareto fronts of IPPO and the PPO_S.

the 4000th episode, which is faster than that of IPPO at the 6000th episode. In addition, without weights as input, the PPO_S training process was more stable than that of IPPO, indicating that the weights of the two objectives caused the training process to fluctuate. However, considering the weights as input can lead to the PPO model being trained to obtain a higher average reward. This may allow the PPO to make better decisions at each point. To prove this hypothesis, we compared IPPO and the PPO_S in 25 instances with

two DDT values. From Table 4, we can see that IPPO all best average IGDs and *P* values with DDT values of 1 and 1.5. Simultaneously, to verify the effectiveness of the two PPO networks in IPPO, we illustrate the Pareto fronts of IPPO and the PPO_S by randomly choosing the two instances shown in Fig. 6. This indicates that the IPPO can obtain a broader Pareto front than the PPO_S in these instances. All IPPO Pareto solutions were better than those of the PPO_S.

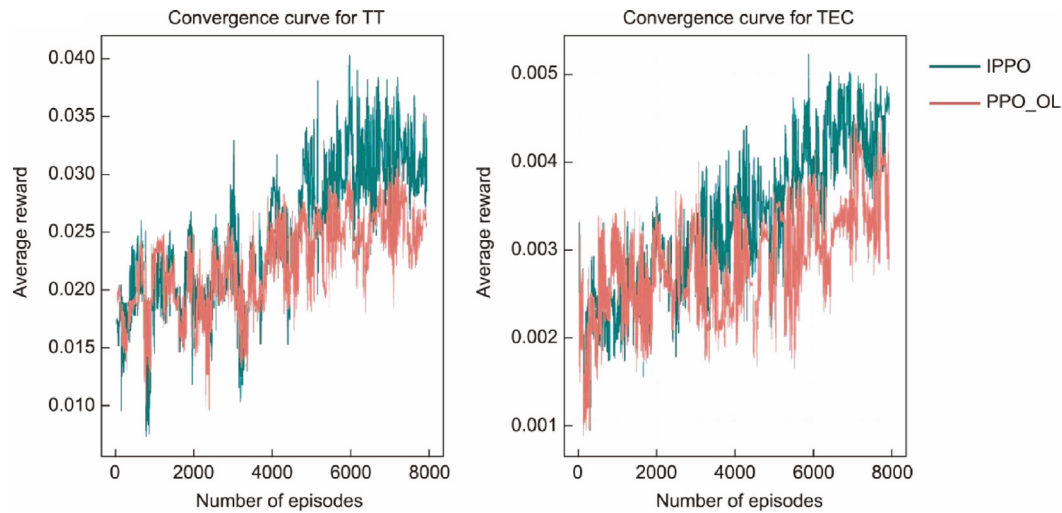


Fig. 7. Convergence curves of IPPO and the PPO_OL.

Table 5
IGD values and *P* values between IPPO and PPO_OL.

Instances	IGD (DDT = 1.0)		<i>P</i> (DDT = 1.0)		IGD (DDT = 1.5)		<i>P</i> (DDT = 1.5)	
	PPO_OL	IPPO	PPO_OL	IPPO	PPO_OL	IPPO	PPO_OL	IPPO
J10	6.94×10^2	5.57×10^2	0.30	0.51	2.41×10^3	9.88×10^1	0.08	0.68
J20	2.00×10^3	3.04×10^2	0.06	0.75	2.29×10^3	4.67×10^2	0.16	0.72
J50	1.89×10^3	7.91×10^2	0.26	0.53	3.10×10^3	3.19×10^2	0.10	0.83
J100	5.26×10^3	2.85×10^3	0.17	0.36	7.15×10^3	2.67×10^3	0.15	0.72
J200	2.71×10^4	1.37×10^4	0.18	0.58	1.74×10^4	3.77×10^3	0.14	0.74

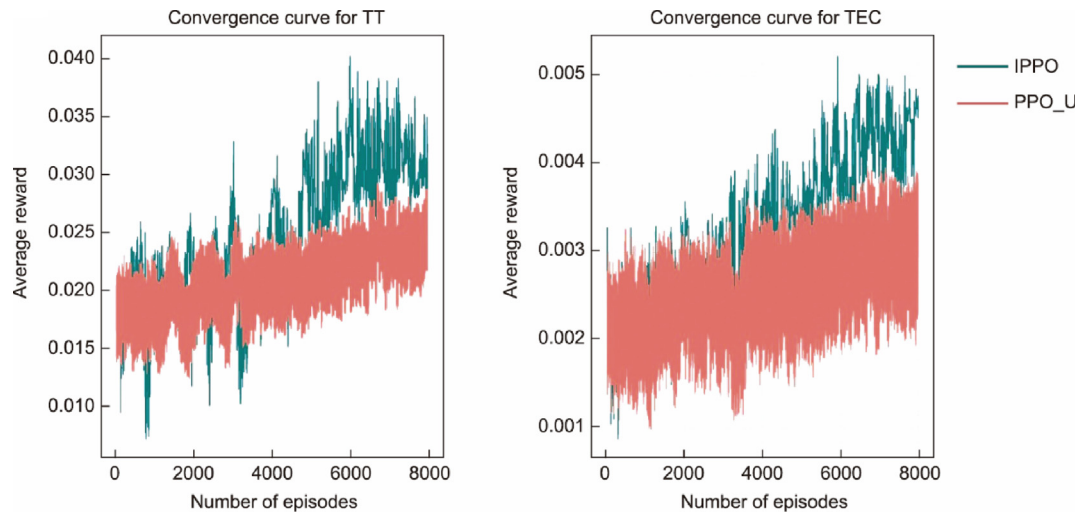


Fig. 8. Convergence curves of IPPO and the PPO_U.

5.2.2. IPPO vs PPO_OL: Validation of the two-stage training

This paper proposes a new learning strategy to use training data more efficiently. This experiment compared IPPO and PPO with the old strategy (PPO_OL). First, Fig. 7 shows that IPPO with the new learning strategy can converge 1000 episodes earlier than the PPO_OL. This indicates that reusing the training data can accelerate the convergence procedure. In addition, it causes the IPPO to converge to a higher average reward of 18% and 27% for TT and TEC, respectively.

We compared IPPO and the PPO_OL in 25 instances with two DDT values to verify the effectiveness of the IPPO model. Table 5 shows that IPPO all best average IGDs and P values with DDT values of 1 and 1.5. Generally, the IGD and P values exhibit the same trend, which means that if the ratio of nondominated solutions generated by IPPO increases, the IGD can perform better than the other methods. The higher IGD of the IPPO indicates that the nondominated solutions generated by the IPPO are closer to the real Pareto front.

5.2.3. IPPO vs PPO_U: Validation of the reward update

In the proposed IPPO method, we improved the reward calculation function. To prove its effect on the IPPO, we compared the method with an improved reward (IPPO) and with an unimproved reward (PPO_U). Both methods used the same training framework and each instance was solved independently 30 times. As shown in Fig. 8, IPPO reaches a stable learning state by approximately 6000 episodes regarding TT and TEC, whereas the PPO_U uses more than 7000 episodes to converge, which indicates that IPPO can converge

faster than the PPO_U with the same training framework. However, the reward training curve of IPPO is not smooth; it rises and drops several times before convergence is reached. In comparison, the curve of the PPO_U was relatively steady, and the overall trend gradually decreased.

We can see from Table 6 that the IPPO obtained best average IGDs and P values with DDT values equal to 1 and 1.5. This indicates that the updated reward in IPPO improves the performance of PPO, including accelerating the convergence and obtaining better solutions for these 50 instances. Detailed analyses between IPPO and PPO_S, PPO_OL and PPO_U can be found in Part E in Appendix A.

5.3. Comparison with dispatch rules

After validating the effectiveness of every improved part in IPPO, this section discusses the comparison of IPPO and several useful dispatching rules containing EDD, modified due date (MDD), minimum slack time (MST), shortest process time (SPT), and largest process time (LPT), and one commonly used MODRL method (abbreviated as SW) [34] using the same PPO network structure as ours in DDT = 1.0. Tables 7 and 8 show that IPPO performs better with IGDs and P values than with dispatching rules and SW. Detailed results can be found in Part F in Appendix A.

5.4. Comparison with metaheuristics and DRL methods

To further investigate the performance of the proposed IPPO, it was compared with the metaheuristics and DRL methods reported

Table 6

IGD values and P values between IPPO and PPO_U.

Instance	IGD (DDT = 1.0)		P (DDT = 1.0)		IGD (DDT = 1.5)		P (DDT = 1.5)	
	PPO_U	IPPO	PPO_U	IPPO	PPO_U	IPPO	PPO_U	IPPO
J10	1.32×10^3	5.57×10^2	0.19	0.51	1.72×10^3	9.88×10^1	0.25	0.68
J20	2.06×10^3	3.04×10^2	0.19	0.75	1.57×10^3	4.67×10^2	0.13	0.72
J50	2.71×10^3	7.91×10^2	0.18	0.53	3.93×10^3	3.19×10^2	0.07	0.83
J100	6.50×10^3	2.85×10^3	0.14	0.36	7.50×10^3	2.67×10^3	0.13	0.72
J200	2.83×10^4	1.37×10^4	0.07	0.58	4.91×10^4	3.77×10^3	0.06	0.74

Table 7

IGD values between IPPO and other rules.

Instance	EDD	MDD	MST	LPT	SPT	SW	IPPO
J10	2.09×10^4	1.58×10^4	1.74×10^4	2.42×10^4	1.45×10^4	1.50×10^4	3.27×10^2
J20	2.41×10^4	1.19×10^4	1.72×10^4	2.12×10^4	1.74×10^4	1.33×10^4	0
J50	2.57×10^4	3.58×10^4	2.92×10^4	2.17×10^4	2.58×10^4	2.18×10^4	6.53×10^3
J100	1.29×10^5	9.11×10^4	7.53×10^4	8.65×10^4	8.97×10^4	4.35×10^4	2.76×10^4
J200	3.15×10^5	2.65×10^5	3.21×10^5	2.20×10^5	2.43×10^5	7.22×10^4	4.64×10^4

Table 8

P values between IPPO and other rules.

Instance	EDD	MDD	MST	LPT	SPT	SW	IPPO
J10	0	0	0	0	0	0	0.90
J20	0	0	0	0	0	0	1.00
J50	0	0	0	0	0	0	0.67
J100	0	0	0	0	0	0	0.53
J200	0	0	0	0	0	0	0.87

Table 9

IGD values between IPPO and other algorithms.

Instances	MOEAD23	Pdjaya23	MAEQL	MAGPI	IPPO
J10	1.48×10^4	1.38×10^4	2.13×10^4	1.53×10^4	3.27×10^2
J20	1.38×10^4	1.47×10^4	2.80×10^4	1.20×10^4	0
J50	1.82×10^4	1.40×10^4	3.72×10^4	1.40×10^4	6.53×10^3
J100	7.20×10^4	7.02×10^4	4.66×10^4	3.37×10^4	2.76×10^4
J200	1.89×10^5	1.50×10^5	6.81×10^4	5.55×10^4	4.64×10^4

in recent publications, including multi-objective evolutionary algorithm based on decomposition 2023 (MOEAD23) [47], Pareto-based discrete jaya algorithm (Pdjaya23) [48], and multi-agent envelope Q-learning (MAEQL) [39]. In addition, IPPO was compared with a multi-agent generalized policy improvement (MAGPI) method, which adopts the generalized policy improvement (GPI) developed in Ref. [49] to train multiple agents under the collaborative framework of this work. To implement MAGPI, baseline GPI codes were adopted from GitHub, and the GPI component was incorporated into the multi-agent framework of this study. The dimensions of the hidden layers of MAEQL and MAGPI were the same as those of IPPO.

As shown in Tables 9 and 10, IPPO performs better in terms of IGDs and *P* values compared with the metaheuristic algorithms and two MODRLs, which indicates the effectiveness of the learning strategy and network design. Detailed results can be found in Part F in Appendix A.

The Pareto fronts shown in Figs. 9 and 10 present the Pareto approximations obtained by the different algorithms on two randomly selected instances. It can be observed that the Pareto fronts generated by IPPO are closer to the bottom-left corner of the figures than those obtained by other algorithms in two-dimensional coordinates, indicating that IPPO obtains a broader Pareto front than the others in these two instances. The distribution of the non-

Table 10
P values between IPPO and other algorithms.

Instance	MOEAD23	Pdjaya23	MAEQL	MAGPI	IPPO
J10	0.05	0.05	0	0	0.90
J20	0	0	0	0	1.00
J50	0	0	0	0.33	0.67
J100	0.10	0	0	0.37	0.53
J200	0	0	0	0.13	0.87

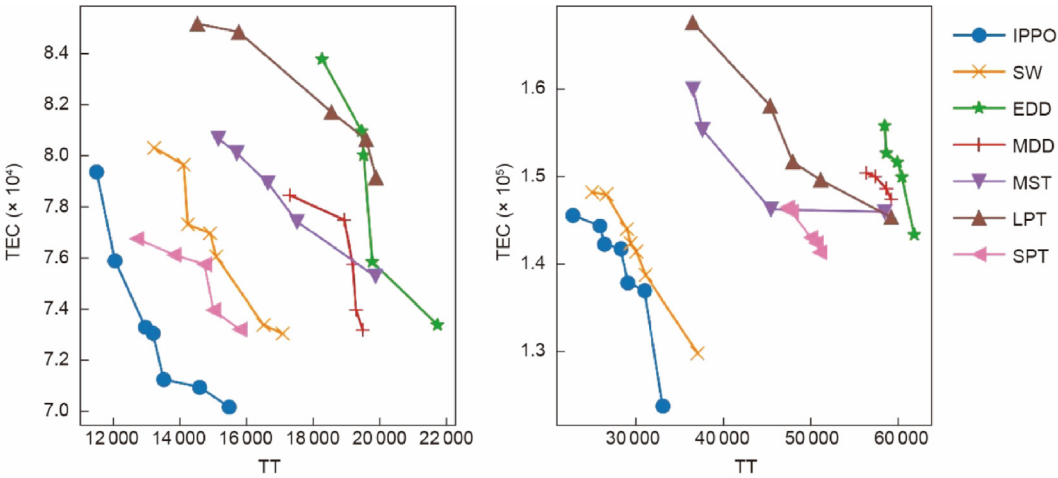


Fig. 9. Pareto fronts of IPPO and compared rules.

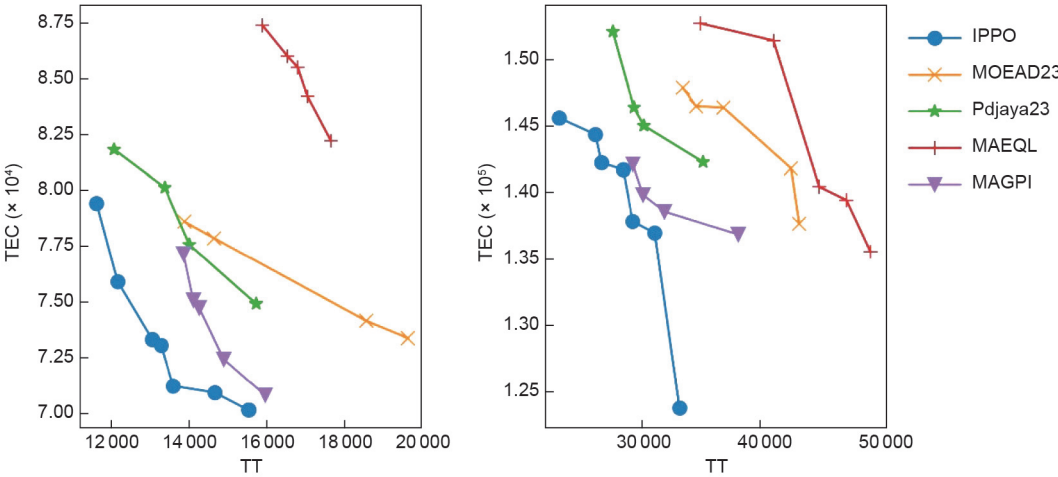


Fig. 10. Pareto fronts of IPPO and compared algorithms.

dominated solutions obtained by IPPO is also better. Compared with the other two DRL algorithms, the results indicate that the proposed method for calculating the loss functions can better optimize both objectives. In addition, the results show that IPPO and MAGPI can obtain better solutions than the other metaheuristic algorithms because global information is considered in the learning process.

6. Conclusions and future work

This study presented a DHHBFSP with TT and TEC criteria. To solve this complex problem efficiently, this study proposes a multi-objective DRL approach based on PPO, called IPPO. We define the state spaces, action spaces, and reward functions for different agents to complete a MOMDP and improve the reward function from three aspects. Then, a new learning strategy is introduced in the IPPO by training the single-policy data and dual-policy data separately to use the old policy data repeatedly. Meanwhile, for agents in the framework, two PPO networks with the same structure and different target sets of weights are defined in a single agent to broaden the Pareto front. Extensive experiments showed that the improved reward function and new learning strategy can speed up the convergence procedure and obtain a better model for most instances. In addition, the two PPO networks allow the distribution of nondominated solutions to be more scattered, although the IPPO convergence process is unstable and sensitive to the training setting. The promising abilities of the proposed IPPO algorithm were demonstrated in comparison with other MODRLs, metaheuristics, and dispatching rules. IPPO cannot only obtain better nondominated solutions in all instances, but also forms a broader Pareto front. This indicates that the proposed loss function calculation method can optimize both objectives better than the other methods.

In future work, we will optimize the training settings to ensure that IPPO performs equally among various instances. In addition, we may consider the applicability of the proposed algorithm to different types of distributed scheduling problems, such as distributed job shop scheduling problems and distributed flexible job shop scheduling problems. Finally, because metaheuristics can achieve better solutions, a new DRL method combining metaheuristics for multi-objective problems will be investigated.

Acknowledgments

This work was partially supported by the National Key Research and Development Program of the Ministry of Science and Technology of China (2022YFE0114200) and the National Natural Science Foundation of China (U20A6004).

Compliance with ethics guidelines

Xueyan Sun, Weiming Shen, Jiaxin Fan, Birgit Vogel-Heuser, Fandi Bi, and Chunjiang Zhang declare that they have no conflict of interest or financial conflicts to disclose.

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.eng.2024.11.033>.

References

- [1] Gao L, Shen W, Li X. New trends in intelligent manufacturing. *Engineering* 2019;5(4):619–20.
- [2] Han W, Guo F, Su X. A reinforcement learning method for a hybrid flow-shop scheduling problem. *Algorithms* 2019;12(11):222.

- [3] Martinez S, Dauzère-Pérès S, Guéret C, Mati Y, Sauer N. Complexity of flowshop scheduling problems with a new blocking constraint. *Eur J Oper Res* 2006;169(3):855–64.
- [4] Srari JS, Kumar M, Graham G, Phillips W, Tooze J, Ford S, et al. Distributed manufacturing: scope, challenges and opportunities. *Int J Prod Res* 2016;54(23):6917–35.
- [5] Shao Z, Pi D, Shao W. Hybrid enhanced discrete fruit fly optimization algorithm for scheduling blocking flow-shop in distributed environment. *Expert Syst Appl* 2020;145:113147.
- [6] Qin HX, Han YY, Liu YP, Li JQ, Pan QK, Han X. A collaborative iterative greedy algorithm for the scheduling of distributed heterogeneous hybrid flow shop with blocking constraints. *Expert Syst Appl* 2022;201:117256.
- [7] Qian F. Smart process manufacturing toward carbon neutrality: digital transformation in process manufacturing for achieving the goals of carbon peak and carbon neutrality. *Engineering* 2023;27(8):1–2.
- [8] Wang R, Jiang L, Wang YD, Roskilly AP. Energy saving technologies and mass-thermal network optimization for decarbonized iron and steel industry: a review. *J Clean Prod* 2020;274:122997.
- [9] He K, Wang L. A review of energy use and energy-efficient technologies for the iron and steel industry. *Renew Sustain Energy Rev* 2017;70:1022–39.
- [10] Hernandez AG, Paoli L, Cullen JM. How resource-efficient is the global steel industry? *Resour Conserv Recycl* 2018;133:132–45.
- [11] Gao Z, Geng Y, Wu R, Chen W, Wu F, Tian X. Analysis of energy-related CO₂ emissions in China's pharmaceutical industry and its driving forces. *J Clean Prod* 2019;223:94–108.
- [12] Ribas I, Companys R, Tort-Martorell X. Efficient heuristics for the parallel blocking flow shop scheduling problem. *Expert Syst Appl* 2017;74:41–54.
- [13] Shao Z, Shao W, Pi D. Effective heuristics and metaheuristics for the distributed fuzzy blocking flow-shop scheduling problem. *Swarm Evol Comput* 2020;59:100747.
- [14] Riedmiller S, Riedmiller M. A neural reinforcement learning approach to learn local dispatching policies in production scheduling. In: *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence*; 1999 Jul 31–Aug 6; Stockholm, Sweden. San Francisco: Morgan Kaufmann Publishers Inc.; 1999. p. 764–71.
- [15] Zhang G, Xing K, Cao F. Discrete differential evolution algorithm for distributed blocking flowshop scheduling with makespan criterion. *Eng Appl Artif Intell* 2018;76:96–107.
- [16] Chen S, Pan QK, Gao L, Sang HY. A population-based iterated greedy algorithm to minimize total flowtime for the distributed blocking flowshop scheduling problem. *Eng Appl Artif Intell* 2021;104:104375.
- [17] Shao Z, Shao W, Pi D. LS-HH: a learning-based selection hyper-heuristic for distributed heterogeneous hybrid blocking flow-shop scheduling. *IEEE Trans Emerg Top Comput Intell* 2023;7(1):111–27.
- [18] Zinn J, Ockier P, Vogel-Heuser B. Deep Q-learning for the control of special-purpose automated production systems. In: *Proceedings of the 2020 IEEE 16th International Conference on Automation Science and Engineering (CASE)*; 2020 Aug 20–21; Hong Kong, China. Piscataway: IEEE; 2020. p. 1434–40.
- [19] Ren J, Ye C, Li Y. A new solution to distributed permutation flow shop scheduling problem based on NASH Q-Learning. *Adv Prod Eng Manag* 2021;16(3):269–84.
- [20] Yang S, Wang J, Xu Z. Real-time scheduling for distributed permutation flowshops with dynamic job arrivals using deep reinforcement learning. *Adv Eng Inform* 2022;54:101776.
- [21] Chen JF, Wang L, Peng ZP. A collaborative optimization algorithm for energy-efficient multi-objective distributed no-idle flow-shop scheduling. *Swarm Evol Comput* 2019;50:100557.
- [22] Zhang X, Liu X, Cichon A, Królczyc G, Li Z. Scheduling of energy-efficient distributed blocking flowshop using pareto-based estimation of distribution algorithm. *Expert Syst Appl* 2022;200:116910.
- [23] Mou J, Duan P, Gao L, Liu X, Li J. An effective hybrid collaborative algorithm for energy-efficient distributed permutation flow-shop inverse scheduling. *Future Gener Comput Syst* 2022;128:521–37.
- [24] Zhao F, Di S, Wang L. A hyperheuristic with Q-learning for the multiobjective energy-efficient distributed blocking flow shop scheduling problem. *IEEE Trans Cybern* 2022;53(5):3337–50.
- [25] Shao Z, Shao W, Chen J, Pi D. MQL-MM: a meta-Q-learning-based multi-objective metaheuristic for energy-efficient distributed fuzzy hybrid blocking flow-shop scheduling problem. *IEEE Trans Evol Comput* 2024;1–1.
- [26] Zhao F, Zhou G, Xu T, Zhu N, Jonrinaldi. A knowledge-driven cooperative scatter search algorithm with reinforcement learning for the distributed blocking flow shop scheduling problem. *Expert Syst Appl* 2023;230:120571.
- [27] Bao H, Pan Q, Ruiz R, Gao L. A collaborative iterated greedy algorithm with reinforcement learning for energy-aware distributed blocking flow-shop scheduling. *Swarm Evol Comput* 2023;83:101399.
- [28] Liu C, Xu X, Hu D. Multiobjective reinforcement learning: a comprehensive overview. *IEEE Trans Syst Man Cybern* 2014;45(3):385–98.
- [29] Gábor Z, Kalmár Z, Szepesvári C. Multi-criteria reinforcement learning. In: *Proceedings of the Fifteenth International Conference on Machine Learning*; 1998 Jul 24–27; Madison, WI, USA. San Francisco: Morgan Kaufmann Publishers; 1998. p. 197–205.
- [30] Feinberg EA, Shwartz A. Constrained Markov decision models with weighted discounted rewards. *Math Oper Res* 1995;20(2):302–20.
- [31] Russell SJ, Zimdars A. Q-decomposition for reinforcement learning agents. In: *Proceedings of the Twentieth International Conference on International*

- Conference on Machine Learning; 2003 Aug 21–24; Washington, DC, USA. Palo Alto: AAAI Press; 2003. p. 656–63.
- [32] Barrett L, Narayanan S. Learning all optimal policies with multiple criteria. In: Proceedings of the 25th international conference on Machine learning; 2008 Jul 5–9; Helsinki, Finland. New York: ACM; 2008. p. 41–7.
- [33] Van Moffaert K, Nowé A. Multi-objective reinforcement learning using sets of pareto dominating policies. *J Mach Learn Res* 2014;15(1):3483–512.
- [34] Mossalam H, Assael YM, Roijers DM, Whiteson S. Multi-objective deep reinforcement learning. 2016. arXiv:1610.02707.
- [35] Abels A, Roijers D, Lenaerts T, Nowé A, Steckelmacher D. Dynamic weights in multi-objective deep reinforcement learning. 2018. arXiv:1809.07803.
- [36] Nguyen TT, Nguyen ND, Vamplew P, Nahavandi S, Dazeley R, Lim CP. A multi-objective deep reinforcement learning framework. *Eng Appl Artif Intell* 2020;96:103915.
- [37] Siddique U, Weng P, Zimmer M. Learning fair policies in multi-objective (deep) reinforcement learning with average and discounted rewards. In: Proceedings of the 37th International Conference on Machine Learning; 2020 Jul 13–18; Vienna, Austria. Brookline: JMLR; 2020. p. 8905–15.
- [38] He Z, Tran KP, Thomassey S, Zeng X, Xu J, Yi C. Multi-objective optimization of the textile manufacturing process using deep-Q-network based multi-agent reinforcement learning. *J Manuf Syst* 2022;62:939–49.
- [39] Yang R, Sun X, Narasimhan K. A generalized algorithm for multi-objective reinforcement learning and policy adaptation. In: Proceedings of the 33rd International Conference on Neural Information Processing Systems; 2019 Dec 8–14; Vancouver, BC, Canada. New York: Curran Associates; 2019. p. 14636–47.
- [40] Luo S, Zhang L, Fan Y. Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. *Comput Ind Eng* 2021;159:107489.
- [41] Lowe R, Wu Y, Tamar A, Harb J, Pieter P, Mordatch I. Multi-agent actor-critic for mixed cooperative-competitive environments. In: Proceedings of the 31st International Conference on Neural Information Processing Systems; 2017 Dec 4–9; Long Beach, CA, USA. New York: Curran Associates; 2017. p. 6382–93.
- [42] Yu C, Velu A, Vinitisky E, Gao J, Wang Y, Bayen A, et al. The surprising effectiveness of PPO in cooperative multi-agent games. In: Proceedings of the 36th International Conference on Neural Information Processing Systems; 2022 Nov 28–Dec 9; New Orleans, LA, USA. New York: Curran Associates; 2024. p. 24611–24.
- [43] Engstrom L, Ilyas A, Santurkar S, Tsipras D, Janoos F, Rudolph L, et al. Implementation matters in deep RL: a case study on PPO and TROP. In: Proceedings of 8th International Conference on Learning Representations, 2020 April 26–30; Addis Ababa, Ethiopia. Appleton: ICLR; 2020. p. 12883–98.
- [44] Sun X, Shen W, Vogel-Heuser B. A hybrid genetic algorithm for distributed hybrid blocking flowshop scheduling problem. *J Manuf Syst* 2023;71:390–405.
- [45] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal policy optimization algorithms. 2017. arXiv:1707.06347.
- [46] Zhao L, Fan J, Zhang C, Shen W, Zhuang J. A DRL-based reactive scheduling policy for flexible job shops with random job arrivals. *IEEE Trans Autom Sci Eng* 2024;21(3):2912–23.
- [47] Zhao F, Zhang H, Wang L, Xu T, Zhu N, Jonrinaldi J. A multi-objective discrete differential evolution algorithm for energy-efficient distributed blocking flow shop scheduling problem. *Int J Prod Res* 2023;62(12):4226–44.
- [48] Zhao F, Zhang H, Wang L. A pareto-based discrete jaya algorithm for multiobjective carbon-efficient distributed blocking flow shop scheduling problem. *IEEE Trans Industr Inform* 2023;19(8):8588–99.
- [49] Alegre LN, Bazzan ALC, Roijers DM, Nowé A, da Silva BC. Sample-efficient multi-objective learning via generalized policy improvement prioritization. 2023. arXiv:2301.07784.